

Applying Taylor Approximation for Optimization.

Current value of variables: w^t

Find a w^{t+1} s.t. $F(w^{t+1}) < F(w^t)$

Express $w^{t+1} = w^t + \lambda \vec{e}$

$$\underline{F(w^{t+1})} = F(w^t + \lambda \vec{e})$$

$$\approx F(w^t) + \nabla F(w^t)^T [w^{t+1} - w^t]$$

$$= \underline{F(w^t)} + \nabla F(w^t)^T \lambda \vec{e}$$

$$F(w^{t+1}) < F(w^t)$$

$$\Rightarrow \nabla F(w^t)^T \lambda \vec{e} < 0$$

choose $\lambda > 0$

$$\Rightarrow \text{choose } \vec{e} \text{ s.t. } \nabla F(w^t)^T \vec{e} < 0$$

Any $\vec{e}$ where $\nabla F(w^t) \cdot \vec{e} < 0$, provides a direction along which if we move from w^t we will reduce the $F(w)$.

But if we choose $\vec{e} =$, the change will

$$\left\{ \begin{array}{l} \|\vec{e}\| = 1 \\ \text{Always possible by choosing} \\ \vec{e} = \frac{w^{t+1} - w^t}{\|w^{t+1} - w^t\|} \quad \lambda = \|w^{t+1} - w^t\| \end{array} \right.$$

Gradient descent algorithm

Input: $F(w)$, learning rate. $\{\eta_t\}$ $t = \text{iteration number}$.

- Choose an arbitrary initial point w^0
- Epoch $t = 0$
- η_t = Chosen learning rate schedule
- While stopping criteria not reached {
 $w^{t+1} \leftarrow w^t - \lambda \nabla F(w^t)$
 if $\nabla F(w^t) \approx 0$, then stop
}

Example: gradient descent

- $F(w_1, w_2) = \frac{1}{2}(w_1^2 + 10w_2^2)$
- $w^0 = [10, 1]^T$ $\nabla F(w^0) = [10, 10]^T$; $\lambda = 0.1$ $F(w^0) = \frac{110}{2}$

$$w^1 = \begin{bmatrix} 10 \\ 1 \end{bmatrix} - 0.1 \begin{bmatrix} 10 \\ 10 \end{bmatrix} = \begin{bmatrix} 9 \\ 0 \end{bmatrix} \quad F(w^1) = \frac{81}{2} \quad \nabla F(w^1) = \begin{bmatrix} 9 \\ 0 \end{bmatrix}$$

$$w^2 = \begin{bmatrix} 9 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} 9 \\ 0 \end{bmatrix} = \begin{bmatrix} 8.1 \\ 0 \end{bmatrix}$$

⋮

$$w^T = \begin{bmatrix} 0.01 \\ 0 \end{bmatrix} \quad \nabla F(w^T) = \left\| \begin{bmatrix} 0.01 \\ 0 \end{bmatrix} \right\| \approx 0 \rightarrow \text{stop}$$

Example: gradient descent

- $F(w_1, w_2) = \frac{1}{2}(w_1^2 + 10w_2^2)$

- $w^0 = [10, 1]^T$

$$w^1 = \begin{bmatrix} 10 \\ 1 \end{bmatrix} - 0.1 \begin{bmatrix} 10 \\ 10 \end{bmatrix} = \begin{bmatrix} 9 \\ 0 \end{bmatrix}$$

$$w^2 = \begin{bmatrix} 9 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} 9 \\ 0 \end{bmatrix} = \begin{bmatrix} 8.1 \\ 0 \end{bmatrix}$$

$\vdots$

$$w^T = \begin{bmatrix} 0.01 \\ 0 \end{bmatrix}$$

$$\nabla F(w^T) = \left\| \begin{bmatrix} 0.01 \\ 0 \end{bmatrix} \right\| \approx 0 \rightarrow \text{stop}$$

$$\nabla F = \begin{bmatrix} w_1 \\ 10w_2 \end{bmatrix}$$

$$\nabla F(w^0) = [10, 10]^T; \quad \eta = 0.1 \quad F(w^0) = \frac{110}{2}$$

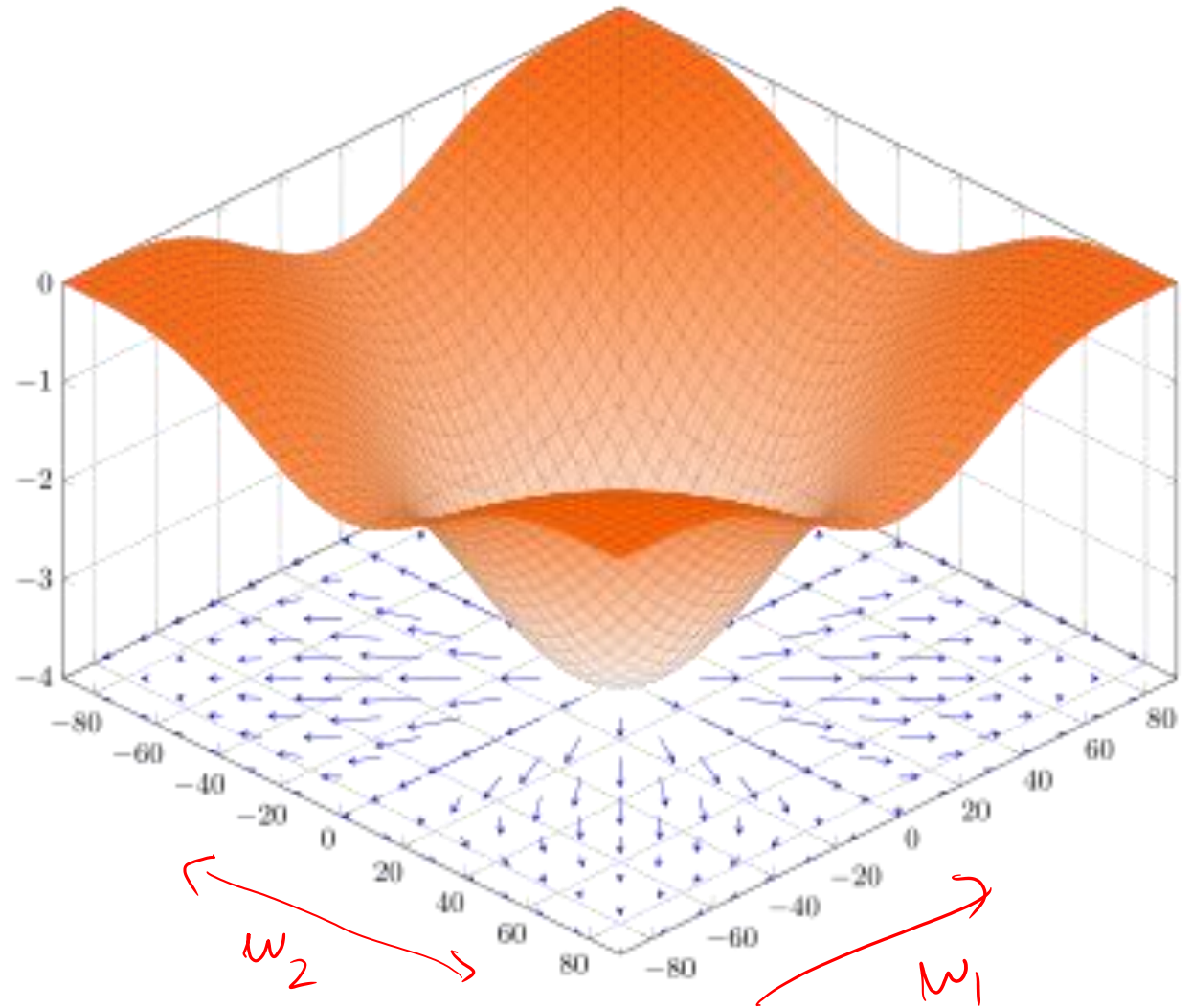
$$F(w^1) = \frac{81}{2}$$

$$\nabla F(w^1) = \frac{(8.1)^2}{2}$$

Visualizing gradients in 2-D using gradient vector fields

Consider function is 2-D

For each point (w_1, w_2) draw a vector where direction is pointing to gradient of function and length of the vector is proportional to the magnitude of the gradient. Thus, vector points to direction of maximum rate of increase of function at that point.



Gradient descent in action.

- See [this colab link](#) to run the demo yourself.

Choosing the learning rate

- Large learning rate $\rightarrow$ oscillations
- Small learning rate $\rightarrow$ several iterations, training slow.

More on learning rates later.

Stochastic gradient descent

- Stochastic approximation to gradient descent optimization when applied over sum of errors on several i.i.d training examples

- Typical training objective:

$$\underline{L(\underline{w})} = \frac{1}{\underline{N}} \sum_{\underline{i}=1}^{\underline{N}} \underline{\ell(f(x^i; \underline{w}), y_i)}$$

- True gradient is expensive to compute when N is large.

$$\nabla L(w) = \frac{1}{N} \sum_{i=1}^N \nabla_w \ell(f(x^i, w), y_i)$$

- Stochastic approximation: select a minibatch $\underline{B} \ll N$ of training samples randomly from the full training set.

$$\nabla L(w) \approx \nabla \tilde{L}_B(w) = \frac{1}{|B|} \sum_{i \in B} \nabla_w \ell(f(x^i, w), y_i)$$

Properties of Stochastic gradient descent (SGD)

- Expected value of the gradient is the true gradient when minibatches are selected randomly.

$$\mathbb{E}_B(\tilde{\nabla} L_B(w)) = \nabla_w L(w)$$

- SDG may require more iterations, but time per iteration is reduced, so overall time reduces.
- Empirical found to be better at optimizing non-convex functions because of noisy nature of gradients.
- For convex functions, under mild conditions with very high probability converges to the local minima if learning rate is decreasing, more precisely satisfies these conditions.

- Sufficient condition for convergence of SGD

Learning rate at iteration T as η_t

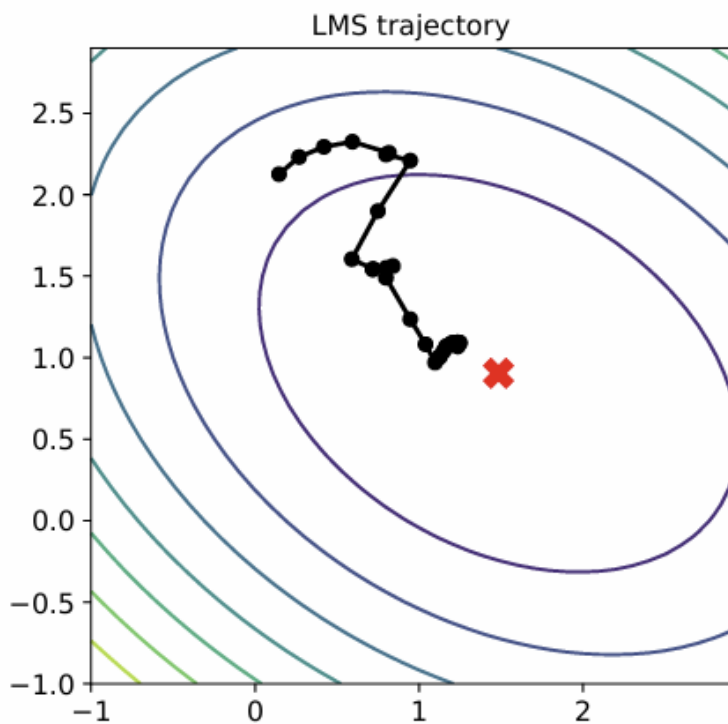
$$t \rightarrow \infty \quad \eta_t \rightarrow 0, \quad \left[\frac{\sum_{t=1}^{\infty} \eta_t^2}{\sum_{t=1}^{\infty} \eta_t} \right] \rightarrow 0$$

Some common examples of learning rate schedules are listed below:

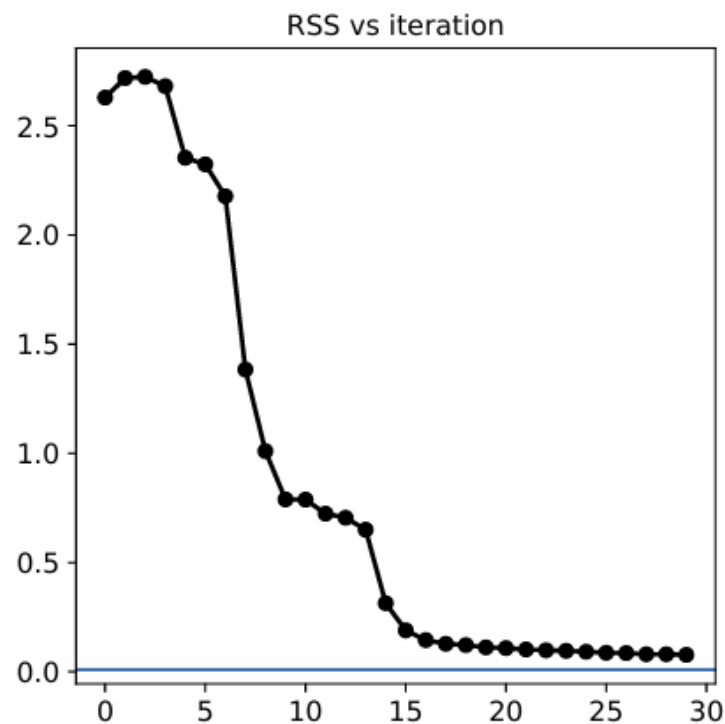
$\eta_t = \eta_i$ if $t_i \leq t \leq t_{i+1}$ piecewise constant ✗

✗ $\eta_t = \eta_0 e^{-\lambda t}$ exponential decay

$\eta_t = \eta_0 (\beta t + 1)^{-\alpha}$ polynomial decay



(a)

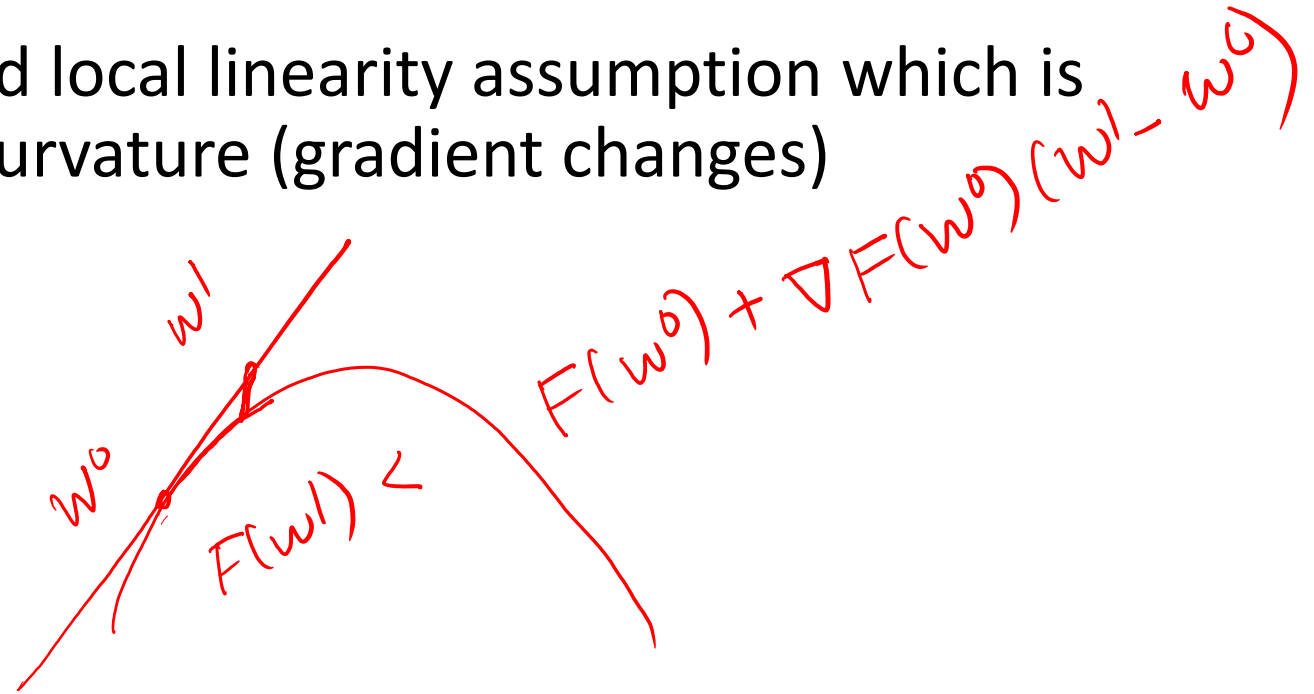
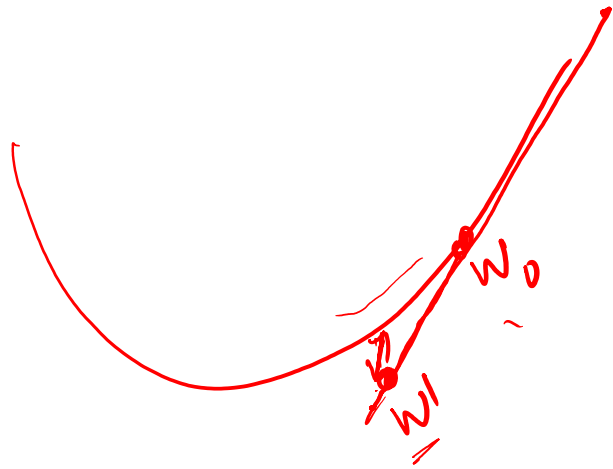


(b)

Figure 8.16: Illustration of the LMS algorithm. Left: we start from $\theta = (-0.5, 2)$ and slowly converging to the least squares solution of $\hat{\theta} = (1.45, 0.93)$ (red cross). Right: plot of objective function over time. Note that it does not decrease monotonically. Generated by [lms demo.ipynb](#).

Second order methods

- GD therefore can only take small jumps $\rightarrow$ too slow. Can we take longer jumps?
- Gradient descent algorithm based local linearity assumption which is violated when the function has curvature (gradient changes)



Second order Taylor series

Rate of change of gradients (curvature) as estimated by the double derivative or Hessian provides another important “local” signal based on which we can take longer jumps.

$$\underline{F(w)} \approx \left[\underline{F(w^0)} + \underline{\nabla F(w^0)^T} (w - w^0) + \frac{1}{2} (w - w^0)^T \underline{H_F(w^0)} (w - w^0) \right]$$

Find w^{t+1} s.t. $\underline{F(w^{t+1})} < \underline{F(w^t)}$ $\tilde{F}_t(w)$

$$\nabla_w \tilde{F}_t(w) = 0$$

$$\underline{\nabla_w} \left[\underline{F(w^t)} + \underline{\nabla F(w^t)^T} (w - w^t) + \frac{1}{2} (w - w^t)^T \underline{H_F(w^t)} (w - w^t) \right]$$

$$= 0 + \nabla F(w^t) + H_F(w^t)(w - w^t) = 0$$
$$\Rightarrow H_F(w^t)w = -\nabla F(w^t) + H_F(w^t)w^t$$

Newton's method

$$\underline{\underline{w}} = H_F(w^t)^{-1} [-\nabla F(w^t) + H_F(w^t) w^t]$$

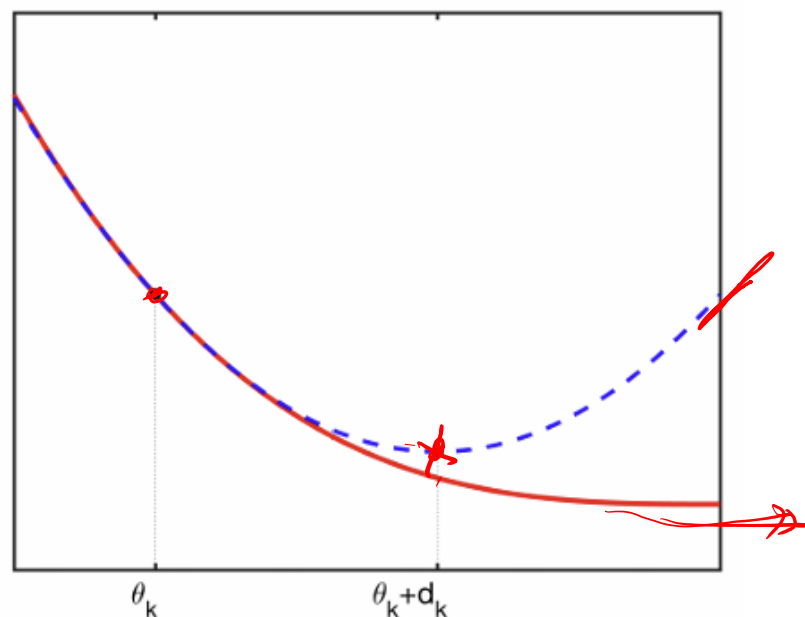
$$\underline{\underline{w^{t+1}}} = \underline{\underline{w^t - H_F(w^t)^{-1} \nabla F(w^t)}}$$

Algorithm 8.1: Newton's method for minimizing a function

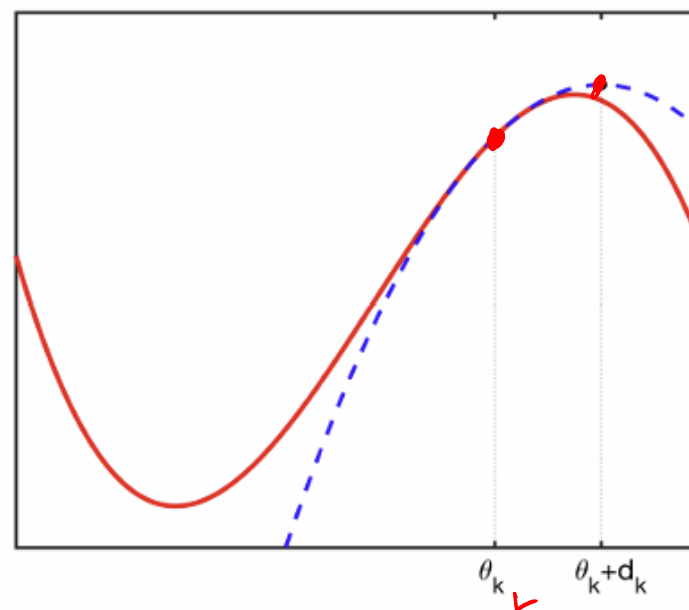
- 1 Initialize θ_0 W^0 $W \rightarrow \theta$ $F \rightarrow \mathcal{L}$
 - 2 **for** $t = 1, 2, \dots$ *until convergence* **do**
 - 3 Evaluate $\underline{g}_t = \nabla \mathcal{L}(\theta_t)$ $\nabla F(W^t)$
 - 4 Evaluate $\mathbf{H}_t = \nabla^2 \mathcal{L}(\theta_t)$
 - 5 Solve $\mathbf{H}_t \underline{d}_t = -\underline{g}_t$ for $\underline{d}_t$
 - 6 Use line search to find stepsize η_t along $\underline{d}_t$ (ignore for now)
 - 7 $\theta_{t+1} = \theta_t + \underline{\eta}_t \underline{d}_t$; $W^{t+1} = W^t - \eta_t H_t^{-1} g_t$
-

Computing Hessian is expensive.

We will discuss semi-second order methods later.



(a)



(b)

w_t

Figure 8.14: Illustration of Newton's method for minimizing a 1d function. (a) The solid curve is the function $\mathcal{L}(x)$. The dotted line $\mathcal{L}_{\text{quad}}(\theta)$ is its second order approximation at θ_t . The Newton step d_t is what must be added to θ_t to get to the minimum of $\mathcal{L}_{\text{quad}}(\theta)$. Adapted from Figure 13.4 of [Van06]. Generated by `newtonsMethodMinQuad.ipynb`. (b) Illustration of Newton's method applied to a nonconvex function. We fit a quadratic function around the current point θ_t and move to its stationary point, $\theta_{t+1} = \theta_t + d_t$. Unfortunately, this takes us near a local maximum of f , not minimum. This means we need to be careful about the extent of our quadratic approximation. Adapted from Figure 13.11 of [Van06]. Generated by `newtonsMethodNonConvex.ipynb`.

Optimizing non-differentiable but continuous functions: subgradient descent algorithm

Examples from ML

Least square regression with L_1 penalty

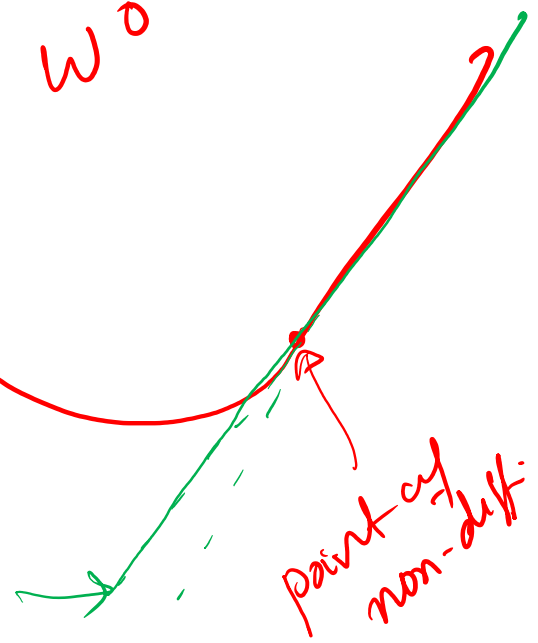
$$\min_w w^T X^T X w - w^T X^T Y + \|w\|_1$$

$$\|w\| = \sum_{j=1}^d |w_j|$$

not-differentiable.

What is subgradient?

$g(w^0)$ is a subgradient of $F(w)$ at w^0
if $\underline{F(w)} \geq \underline{F(w^0)} + (w - w^0)^T \underline{g(w^0)}$



Example

$$F(w) = \max(0, 1-w)$$

One subgradient at $w^0 = 1$

$$\textcircled{1} \quad g(w^0) = 0$$

Second subgradient, at $w^0 = 1$

$$g(w^0) = -1$$

$$F(w) \geq 0 + \underline{g(w^0)}(w-1)$$

$$\textcircled{1} \quad F(w) \geq 0$$

$$\textcircled{2} \quad F(w) \geq -1(w-1)$$

If $g_1(w^0)$ & $g_2(w^0)$ are two subgradients of $F(w)$
then all convex combinations $\lambda g_1(w^0) + (1-\lambda)g_2(w^0)$
will be subgradients for $0 \leq \lambda \leq 1$.

